

Adding a PostgreSQL replica

In order to add a PostgreSQL replica to the Patroni cluster, please refer to [patroni-management.md](#).

Other examples here are largely based on taking a basebackup manually.

Using `pg_basebackup` to resync a replica

If a replica gets out of sync or fails for some reason, you have several options to recover:

1. Run `pg_basebackup` from the secondary
2. Use WAL-G (or older WAL-E) to restore a backup and catch up from there
3. Take a disk snapshot of the primary and clone it on the secondary. Be sure to drop replication slots on the secondary after it comes up.

Running `pg_basebackup`

All commands are supposed to be run on the replica in question if not stated otherwise.

`$upstream` is the hostname of the upstream database box we want to use to resync. This is not necessarily the primary as we can (and should!) use another secondary for this purpose.

1. Stop PostgreSQL: `gitlab-ctl stop postgresql`
2. Backup `postgresql.auto.conf` for later reference: `cp /var/opt/gitlab/postgresql/data/postgresql.auto.conf /var/opt/gitlab/postgresql/postgresql.auto.conf.$(date +%F)`
3. Remove old data directory `mv /var/opt/gitlab/postgresql/data{,.bak}`
4. Connect to `$upstream` and create physical replication slot on a `gitlab-psql` session: `select pg_create_physical_replication_slot($slot)` with `$slot` being a random string you define
5. Start `pg_basebackup` process (preferably in `tmux` session as this takes a while): `sudo -u gitlab-psql PGSSLMODE=disable /opt/gitlab/embedded/bin/pg_basebackup -D /var/opt/gitlab/postgresql/data --slot=$slot -c fast -X stream -P --host=$upstream -p 5432 --username=gitlab-replicator -R`
6. Once finished, review `postgresql.auto.conf` and compare with backup in `/var/opt/gitlab/postgresql/postgresql.auto.conf.$(date +%F)`. Check upstream is `$upstream`.
7. Start PostgreSQL: `gitlab-ctl start postgresql`
8. Let the new replica catch up and become in-sync with `$upstream`.
9. If `$upstream` was a secondary, reconfigure to use the primary and also drop the replication slot created in (5) by connecting to `$upstream` and perform a `select pg_drop_replication_slot($slot)` there

10. Persist `postgresql.auto.conf` configuration in Chef using `gitlab-server::postgresql-standby` recipe (example).
11. Make sure to clean up and remove the old data directory in `/var/opt/gitlab/postgresql/data.bak`

After having started PostgreSQL again, there are the following phases:

1. Crash recovery (during this time, the secondary is not accessible: FATAL: the database system is starting up)
2. Catchup with upstream (secondary is accessible but lags behind)

Useful things to look at:

1. `/var/log/gitlab/postgresql/current`
2. `select * from pg_stat_replication on $upstream`

Here's an example of a `postgresql.auto.conf` that has both streaming replication and archive recovery enabled (note `$upstream`, `$fqdn`, `$slot` need to be replaced):

```
primary_conninfo = 'user=gitlab-replicator password=REDACTED host='$upstream' port=5432 fall
recovery_target_timeline = 'latest'
primary_slot_name = $slot
```

```
restore_command = '/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-fetch -p 32 "%l
```